

ARTIFICIAL INTELLIGENCE

Lecture Set 17
Evaluation Metrics

Dr. Maria Siddiqua
Assistant Professor
Department of AI & DS
FAST-NUCES Karachi
maria.siddiqua@nu.edu.pk

Course Contents

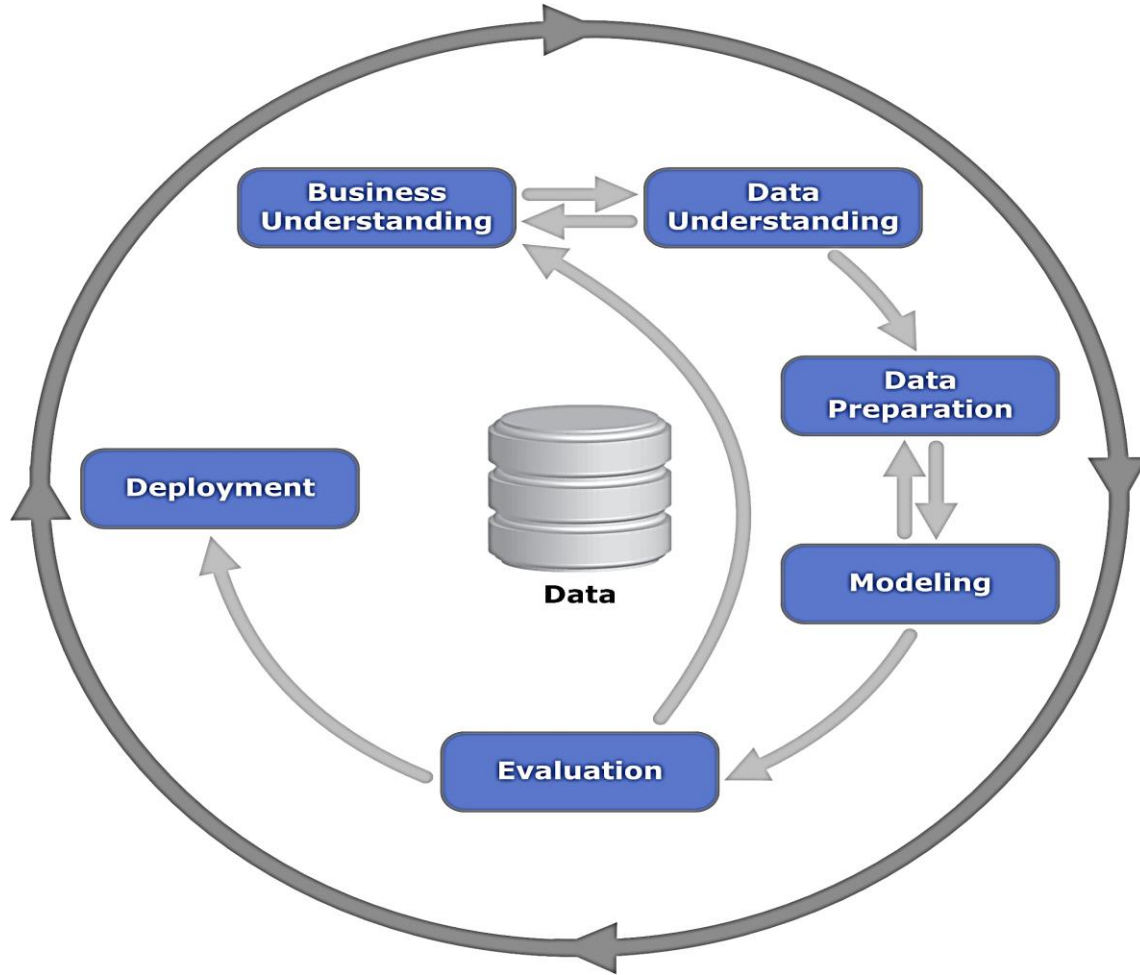
1. Introduction to AI
2. Agents and Environments
3. Problem Solving by Searching
4. Uninformed Search
5. Informed Search
6. Local Search
7. Constraint Satisfaction Problems
8. Adversarial Search
9. Knowledge Representation and Logical Agents
10. Uncertainty, Conditional Probability, Bayes Rule
11. Bayesian Networks, Naïve Bayes Classifier
12. Hidden Markov model, Markov Decision Process

➔ **13. Machine Learning (Supervised, Unsupervised, Reinforcement Learning)**

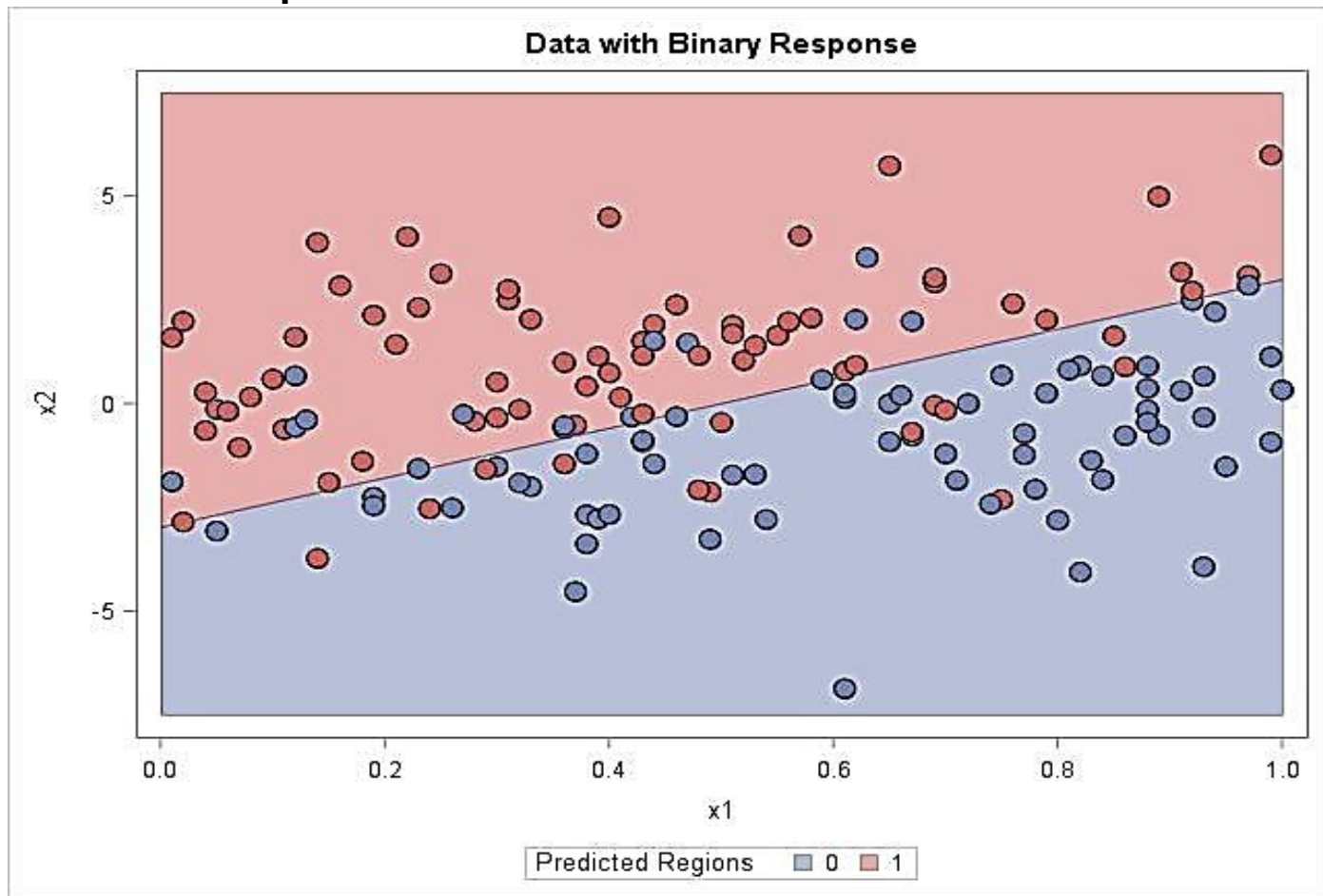
Table of Contents

1. Model Development Process
2. Classification problems
3. Why evaluation metrics?
4. Confusion matrix
5. Accuracy
6. Precision
7. Recall
8. ROC curve

Model Development Process



Classification problems



Classification problems

- X is Input
- Y is binary Output (0/1)
- Model is $\hat{y} = h(X)$
- Two types of models
 - Models that output a categorical class directly (K Nearest neighbor, Decision tree)
 - Models that output a real valued score (SVM, Logistic Regression)
 - Score could be margin (SVM), probability (Logistic Regression, NN)
 - Need to pick a threshold
 - We focus on this type (the other type can be interpreted as an instance)

Why evaluation metrics?

- Training objective (cost function) is only a proxy for real world objective.
- Metrics help capture a business goal into a quantitative target (not all errors are equal).
- Helps organize ML team effort towards that target.
 - Generally in the form of improving that metric on the dev set.
- Useful to quantify the “gap” between:
 - Desired performance and baseline (estimate effort initially).
 - Desired performance and current performance.
 - Measure progress over time (No Free Lunch Theorem).
- Ideally training objective should be the metric, but not always possible. Still, metrics are useful and important for evaluation.

Methods of Estimation

- Holdout
 - Reserve $2/3$ for training and $1/3$ for testing
- Random subsampling
 - Repeated holdout
- Cross validation
 - Partition data into k disjoint subsets – k -fold: train on $k-1$ partitions, test on the remaining one
 - Leave-one-out: $k=n$
- Bootstrap – Sampling with replacement

Problem Definition – Confusion Matrix

- Consider the confusion matrix given below for a binary classifier predicting the presence of a disease
- The classifier made a total of 150 predictions. Out of those 150 cases, the classifier predicted "yes" 100 times, and "no" 50 times.
- In reality, 100 patients in the sample have the disease, and 50 patients do not.

	Predicted No	Predicted Yes
Actual No	45	5
Actual Yes	5	95

- **Accuracy:** Overall, how often is the classifier correct?

$$\begin{aligned} \text{Accuracy} &= \frac{TN+TP}{TN+FP+FN+TP} \\ &= \frac{45 + 95}{150} = \underline{93.33\%} \end{aligned}$$

- **Misclassification Rate:** Overall, how often is it wrong?

$$\begin{aligned} \text{Misclassification Rate} &= \frac{FN+FP}{TN+FP+FN+TP} \\ &= \frac{5 + 5}{150} = \underline{6.67\%} \end{aligned}$$

- **True Positive Rate:** When it's actually yes, how often does it predict yes?

- also known as "Sensitivity" or "Recall"

$$\begin{aligned} \text{True Positive rate} &= \frac{TP}{\text{Actual Yes}} \\ &= \frac{95}{100} = \underline{95\%} \end{aligned}$$

	Predicted No	Predicted Yes
Actual No	45	5
Actual Yes	5	95

- **False Positive Rate:** When it's actually no, how often does it predict yes?

$$\begin{aligned} \text{False Positive rate} &= \frac{FP}{\text{Actual No}} \\ &= \frac{5}{50} = \underline{10\%} \end{aligned}$$

- **True Negative Rate:** When it's actually no, how often does it predict no?

- also known as "Specificity"

$$\begin{aligned} \text{True Negative rate} &= \frac{TN}{\text{Actual No}} \\ &= \frac{45}{50} = \underline{90\%} \end{aligned}$$

- **Precision:** When it predicts yes, how often is it correct?

$$\begin{aligned} \text{Precision} &= \frac{TP}{\text{Predicted Yes}} \\ &= \frac{95}{100} = 95\% \end{aligned}$$

- **Prevalence:** How often does the yes condition actually occur in our sample?

$$\begin{aligned} \text{Prevalence} &= \frac{\text{Actual Yes}}{\text{Total}} \\ &= \frac{100}{150} = \underline{66.67\%} \end{aligned}$$

Score based models

Score = 1



Score = 0

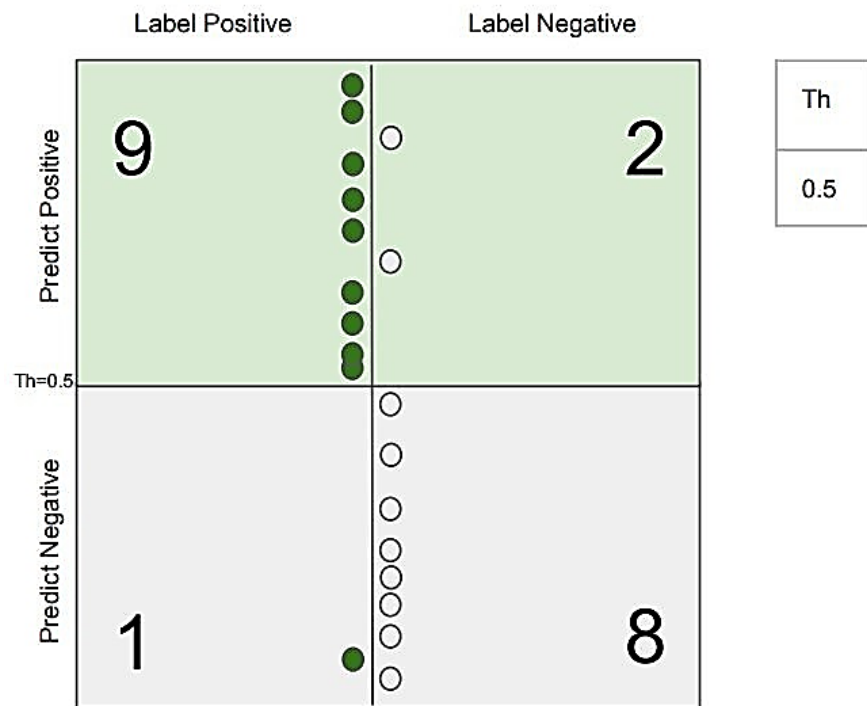
●	Positive labelled example
○	Negative labelled example

$$\text{Prevalence} = \frac{\text{\#positives}}{\text{\#positives} + \text{\#negatives}}$$

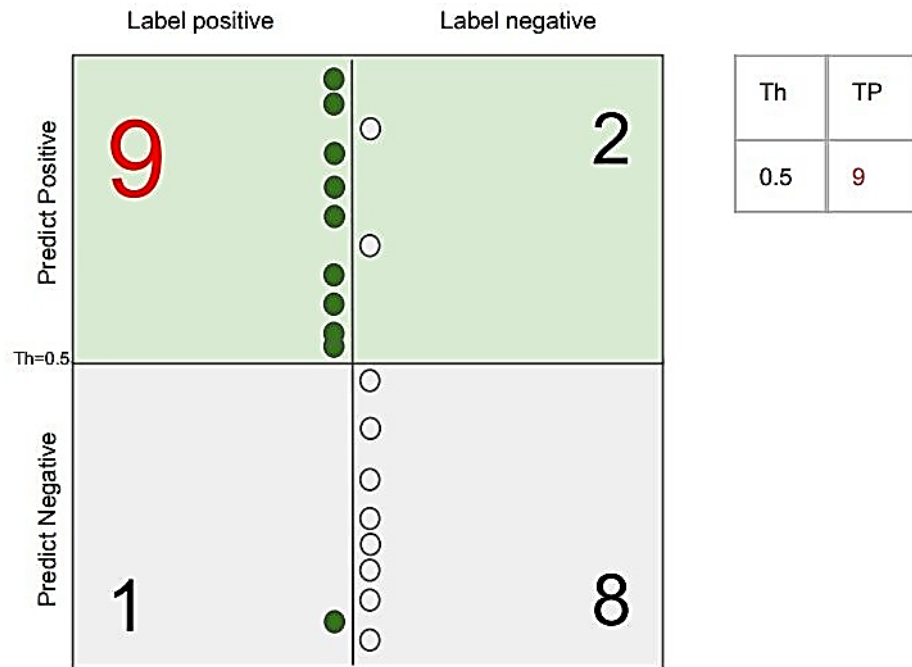
Score based models : Classifier



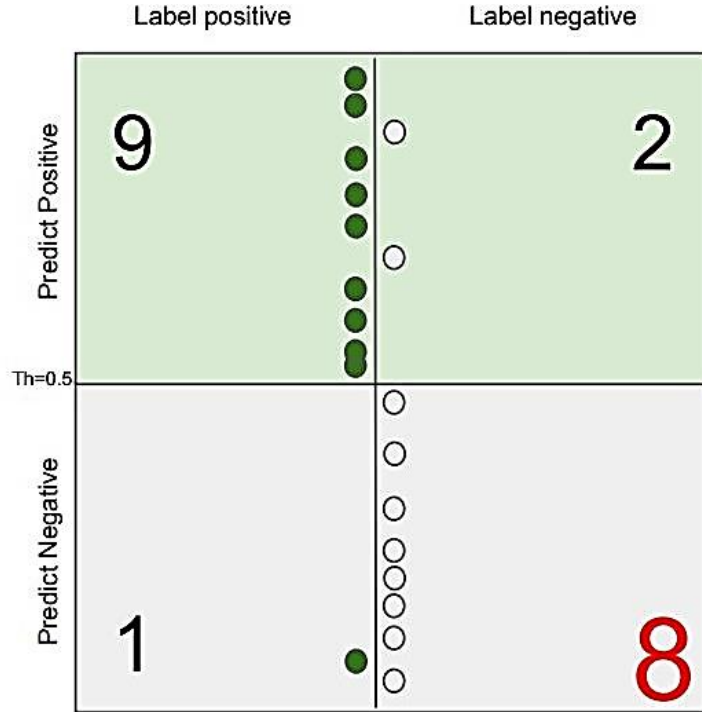
Point metrics: Confusion matrix



Point metrics: True Positives

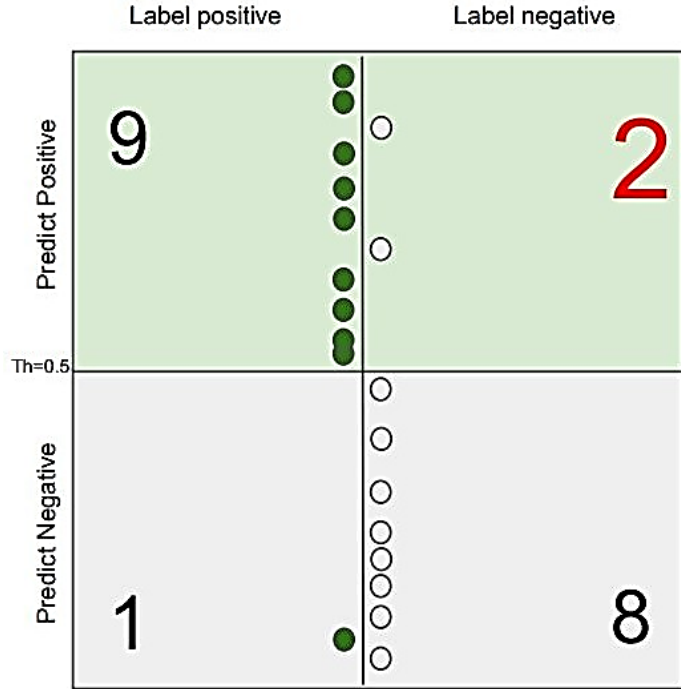


Point metrics: True Negatives



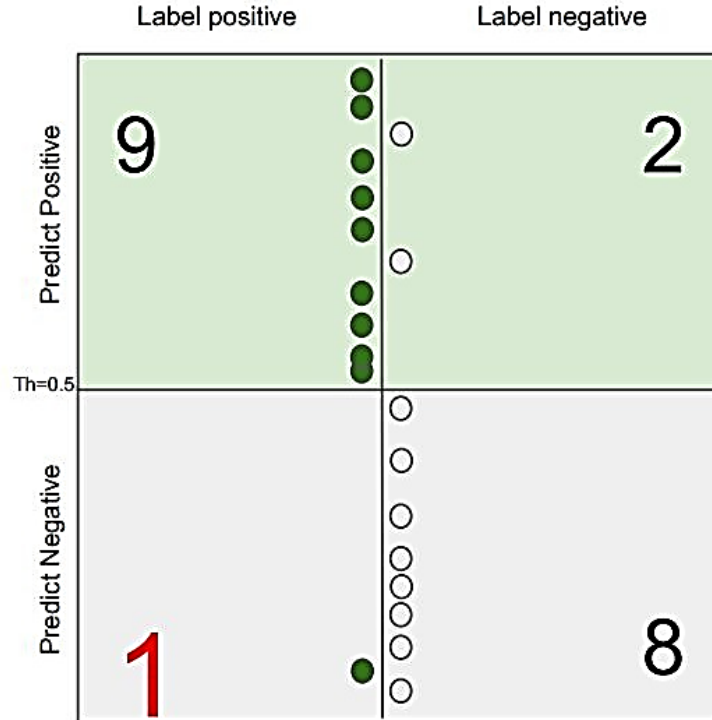
Th	TP	TN
0.5	9	8

Point metrics: False Positives



Th	TP	TN	FP
0.5	9	8	2

Point metrics: False Negatives



Th	TP	TN	FP	FN
0.5	9	8	2	1

Point metrics: Accuracy

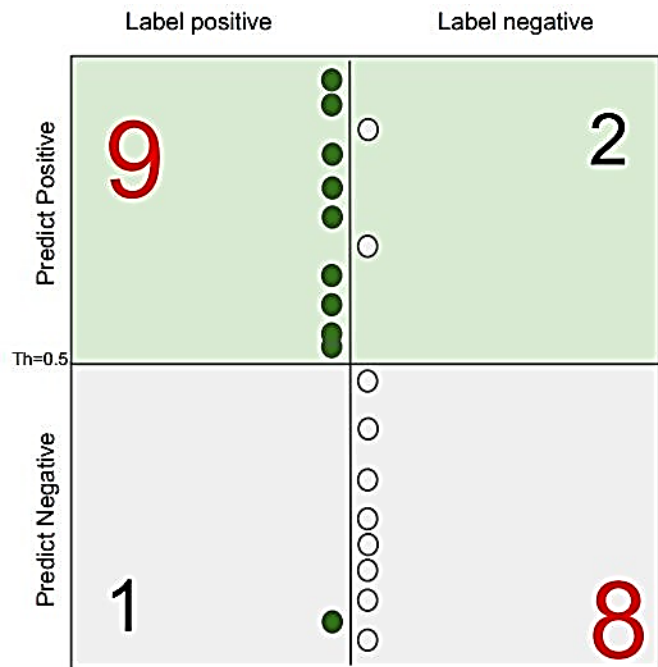
Accuracy = correct predictions/ all predictions

Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$

Point metrics: Accuracy

Accuracy = correct predictions/ all predictions

Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$



Th	TP	TN	FP	FN	Acc
0.5	9	8	2	1	.85

Limitation of Accuracy

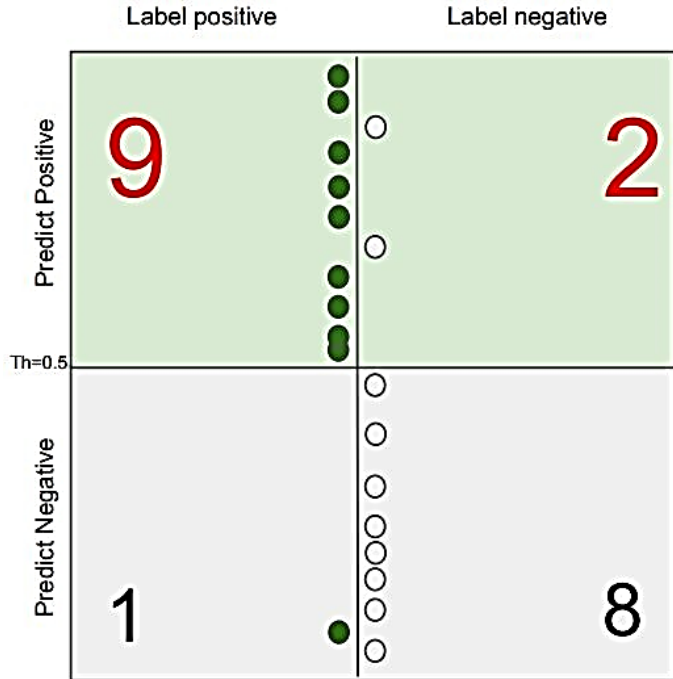
- Consider a 2-class problem
 - Number of Class 0 examples = 9990
 - Number of Class 1 examples = 10
- If model predicts everything to be class 0, accuracy is $9990/10000 = 99.9\%$

Accuracy is misleading because model does not detect any class 1 example

Point metrics: Precision

Precision = $\text{CorrectPositivePredictions} / \text{AllPositivePredictions}$

Precision = $TP / TP + FP$

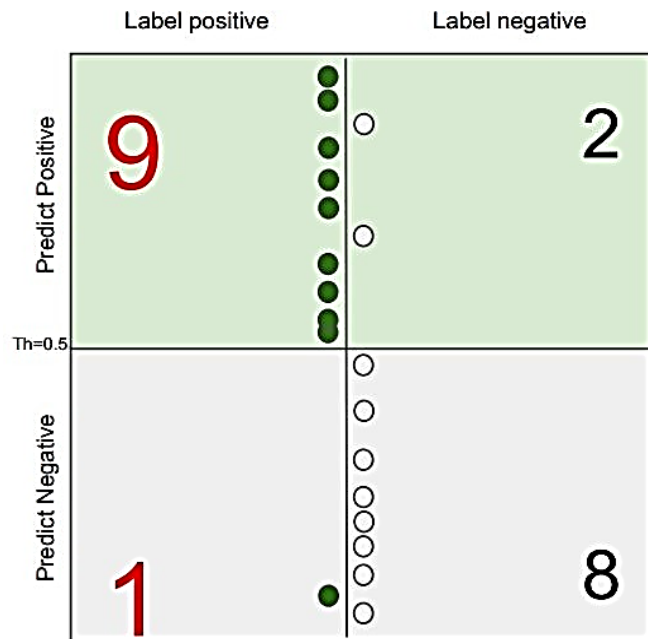


Th	TP	TN	FP	FN	Acc	Pr
0.5	9	8	2	1	.85	.81

Point metrics: Positive Recall (Sensitivity)

Positive Recall: $\text{CorrectPositivePredictions} / \text{AllPositiveSamples}$

Positive Recall = $TP / TP + FN$

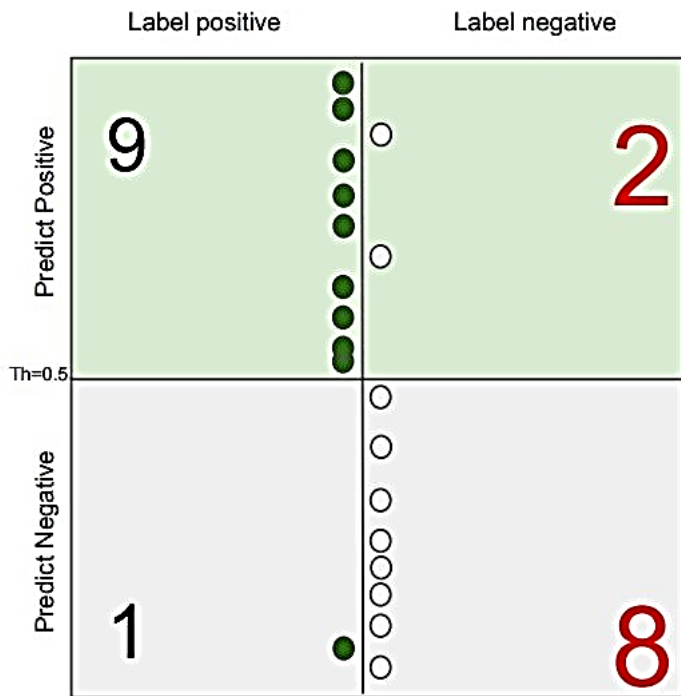


Th	TP	TN	FP	FN	Acc	Pr	Recall
0.5	9	8	2	1	.85	.81	.9

Point metrics: Negative Recall (Specificity)

Negative Recall: $\text{CorrectNegativePredictions} / \text{AllNegativeSamples}$

Negative Recall = $\text{TN} / \text{TN} + \text{FP}$

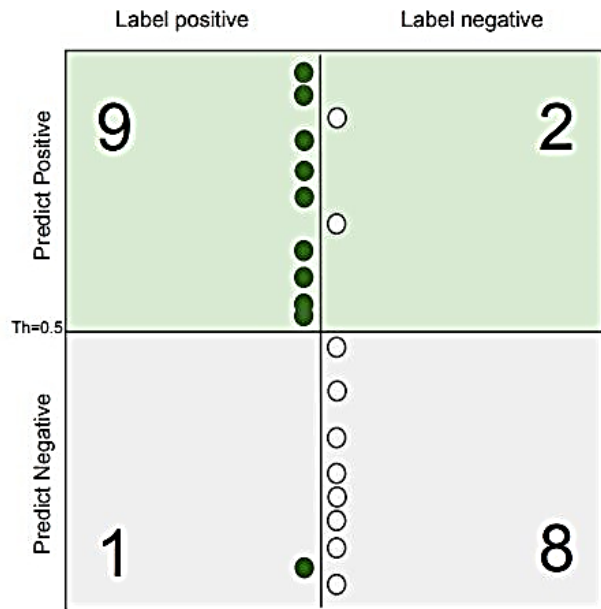


Th	TP	TN	FP	FN	Acc	Pr	Recall	Spec
0.5	9	8	2	1	.85	.81	.9	0.8

Point metrics: F-measure/ F-score

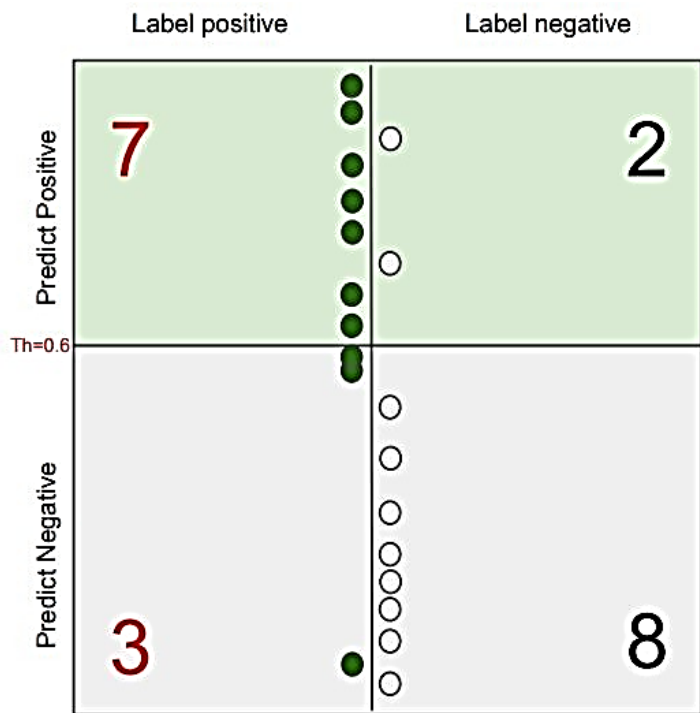
(Harmonic) mean of recall and precision

F-score = $2 \times (\text{precision} \times \text{recall} / (\text{precision} + \text{recall}))$



Th	TP	TN	FP	FN	Acc	Pr	Recall	Spec	F1
0.5	9	8	2	1	.85	.81	.9	.8	.857

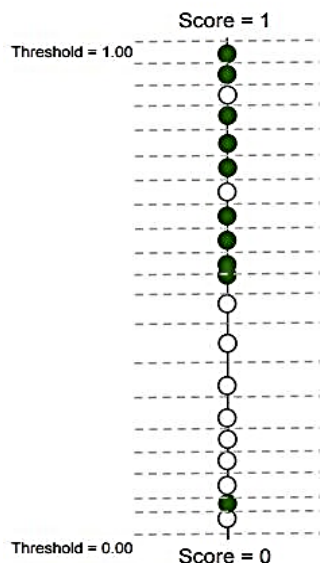
Point metrics: changing the threshold



Th	TP	TN	FP	FN	Acc	Pr	Recall	Spec	F1
0.6	7	8	2	3	.75	.77	.7	.8	.733

Threshold scanning

Threshold Scanning



Threshold	TP	TN	FP	FN	Accuracy	Precision	Recall	Specificity	F1
1.00	0	10	0	10	0.50	1	0	1	0
0.95	1	10	0	9	0.55	1	0.1	1	0.182
0.90	2	10	0	8	0.60	1	0.2	1	0.333
0.85	2	9	1	8	0.55	0.667	0.2	0.9	0.308
0.80	3	9	1	7	0.60	0.750	0.3	0.9	0.429
0.75	4	9	1	6	0.65	0.800	0.4	0.9	0.533
0.70	5	9	1	5	0.70	0.833	0.5	0.9	0.625
0.65	5	8	2	5	0.65	0.714	0.5	0.8	0.588
0.60	6	8	2	4	0.70	0.750	0.6	0.8	0.667
0.55	7	8	2	3	0.75	0.778	0.7	0.8	0.737
0.50	8	8	2	2	0.80	0.800	0.8	0.8	0.800
0.45	9	8	2	1	0.85	0.818	0.9	0.8	0.857
0.40	9	7	3	1	0.80	0.750	0.9	0.7	0.818
0.35	9	6	4	1	0.75	0.692	0.9	0.6	0.783
0.30	9	5	5	1	0.70	0.643	0.9	0.5	0.750
0.25	9	4	6	1	0.65	0.600	0.9	0.4	0.720
0.20	9	3	7	1	0.60	0.562	0.9	0.3	0.692
0.15	9	2	8	1	0.55	0.529	0.9	0.2	0.667
0.10	9	1	9	1	0.50	0.500	0.9	0.1	0.643
0.05	10	1	9	0	0.55	0.526	1	0.1	0.690
0.00	10	0	10	0	0.50	0.500	1	0	0.667

ROC (Receiver Operating Characteristic)

- Developed in 1950s for signal detection theory to analyze noisy signals
 - Characterize the trade-off between positive hits and false alarms
- **ROC** curve plots **TPR** (on the **y**-axis) against **FPR** (on the **x**-axis)

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

	PREDICTED CLASS		
		Yes	No
Actual	Yes	a (TP)	b (FN)
	No	c (FP)	d (TN)

ROC (Receiver Operating Characteristic)

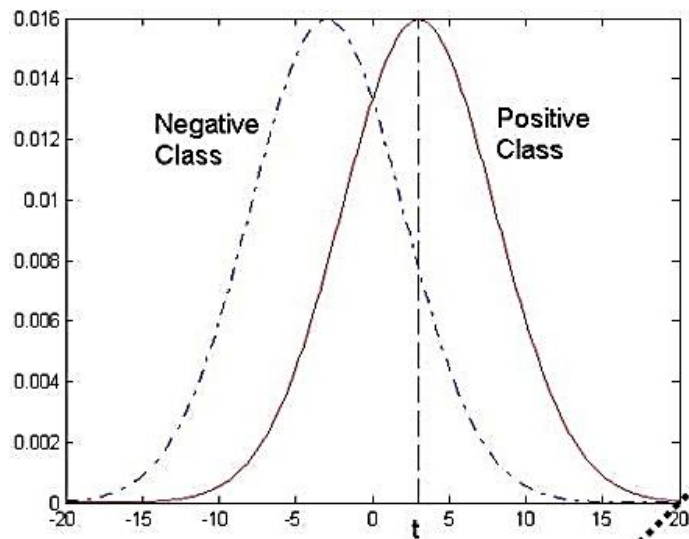
- Performance of each classifier represented as a point on the ROC curve
 - **changing the threshold of algorithm, sample distribution or cost matrix changes the location of the point**

<https://www.youtube.com/watch?v=yvluKK-HFxo>

<https://www.youtube.com/watch?v=GgDTecCcCVE>

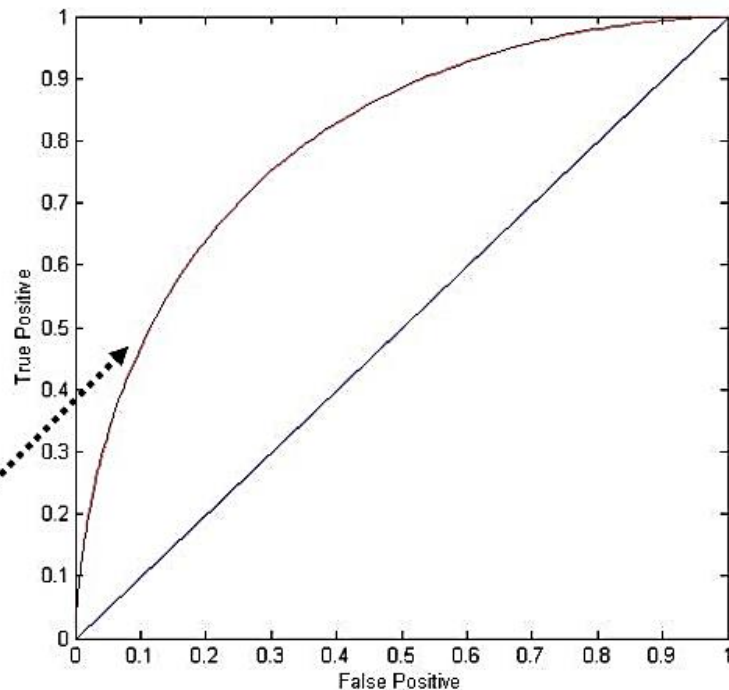
ROC Curve

- 1-dimensional data set containing 2 classes (*positive* and *negative*)
- any points located at $x > t$ is classified as *positive*



At threshold t :

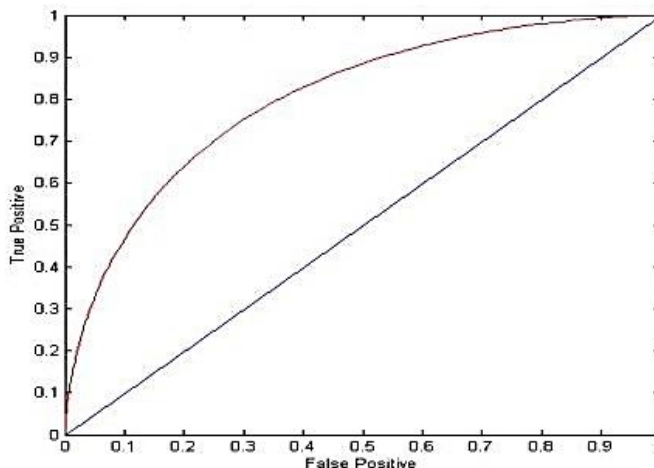
TP=0.5, FN=0.5, FP=0.12, FN=0.88



ROC Curve

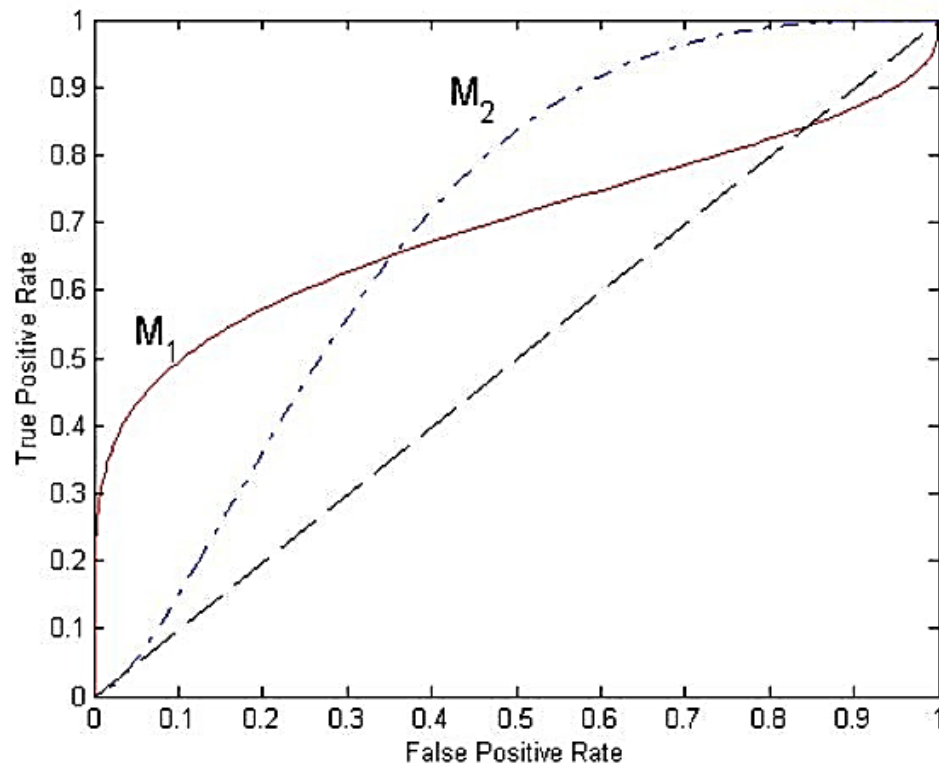
(TP,FP):

- (0,0): declare everything to be negative class
- (1,1): declare everything to be positive class
- (1,0): ideal
- Diagonal line:
 - Random guessing
 - Below diagonal line:
 - prediction is opposite of the true class



	PREDICTED CLASS		
		Yes	No
Actual	Yes	a (TP)	b (FN)
	No	c (FP)	d (TN)

Using ROC for Model Comparison



- No model consistently outperform the other
 - M_1 is better for small FPR
 - M_2 is better for large FPR
- Area Under the ROC curve
 - Ideal: Area = 1
 - Random guess:
 - Area = 0.5

How to Construct an ROC curve

Instance	$P(+ A)$	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.85	-
5	0.85	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

- Use classifier that produces posterior probability for each test instance $P(+|A)$
- Sort the instances according to $P(+|A)$ in decreasing order
- Apply threshold at each unique value of $P(+|A)$
- Count the number of TP, FP, TN, FN at each threshold
- TP rate, $TPR = TP/(TP+FN)$
- FP rate, $FPR = FP/(FP + TN)$

<https://www.youtube.com/watch?v=0awZbJTwgMM>

How to construct an ROC curve

Class	+	-	+	-	-	-	+	-	+	+	
Threshold $\geq$	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4	3	3	3	3	2	2	1	0
FP	5	5	4	4	3	2	1	1	0	0	0
TN	0	0	1	1	2	3	4	4	5	5	5
FN	0	1	1	2	2	2	2	3	3	4	5
TPR	1	0.8	0.8	0.6	0.6	0.6	0.6	0.4	0.4	0.2	0
FPR	1	1	0.8	0.8	0.6	0.4	0.2	0.2	0	0	0

